

Práctica 2: El robot lógico

Lógica proposicional y SAT para un robot doméstico

En esta práctica el robot decide sus movimientos mediante **lógica proposicional** en vez de búsqueda: se codifica la física del robot (posición, acciones, paredes) como una base de conocimiento (KB) en forma normal conjuntiva (CNF) y se pregunta a un solver SAT si existe una asignación de variables — un *modelo* — que haga cierto “el robot llega al objetivo”. Dos de las preguntas (4 y 5) resuelven **el mismo problema** que la Práctica 1 (llegar a un punto, catalogar objetos), pero por este camino distinto: se pide explícitamente comparar ambos enfoques al final.

Se trabajará sobre el fichero `practica2_enunciado.py`, proporcionado parcialmente implementado. Se asume familiaridad con la API de `HouseWorld/Robot` (documentada en `practica0_intro.tex`) y con `SearchProblem` de la Práctica 1 (útil para la comparación final, no necesaria para el código).

1. Entrega

La entrega se realizará en el aula, durante la clase de prácticas del 13 de noviembre de 2026, a través de la tarea habilitada para ello en el aula virtual.

El día de la entrega, una vez entregada la práctica, se cumplimentará en el aula, en modo examen y sin acceso al código, un breve cuestionario en el que se pedirá explicar cómo se ha resuelto alguna cuestión concreta de la solución entregada (ver Sección 4 para un ejemplo del estilo de pregunta que cabe esperar).

2. Infraestructura proporcionada

Todo lo de esta sección ya está implementado (en `sat_solver.py` o en `practica2_enunciado.py`, según se indica): no es TODO.

2.1. `sat_solver.py`: lógica proposicional y SAT

Fichero aparte para no mezclar esta infraestructura con el código propio de la práctica. No usa ninguna librería externa: incluye su propio solver SAT.

<code>Expr</code>	Fórmula proposicional con operadores de Python: <code>~a</code> (NO), <code>a&b</code> (Y), <code>a b</code> (O), <code>a>>b</code> (IMPLICA), <code>a%b</code> (SI Y SOLO SI). Una hoja es <code>Expr('Nombre')</code> .
<code>symbols(*nombres)</code>	Azúcar sintáctico: <code>symbols('A','B') → (Expr('A'), Expr('B'))</code> .
<code>pl_true(expr, model)</code>	Evalúa <code>expr</code> bajo una asignación <code>model: Dict[str,bool]</code> .
<code>to_cnf(expr) → List[List[str]]</code>	Convierte cualquier <code>Expr</code> a CNF: lista de cláusulas, cada cláusula una lista de literales.
<code>find_model(clauses) → Optional[Dict[str,bool]]</code>	El solver SAT (DPLL): busca una asignación que satisfaga todas las cláusulas, o <code>None</code> si es insatisfacible.
<code>negate_literal(lit)</code>	$X \rightarrow \neg X$; $\neg X \rightarrow X$.

Dos formas de trabajar con fórmulas, para dos momentos distintos de la práctica:

- **Álgebra simbólica** (`Expr + to_cnf`): cómoda de leer y escribir, pero convertir una fórmula grande a CNF es caro. Se usa **solo en la Pregunta 1** (calentamiento).
- **Cláusulas directas** (`List[List[str]]`): un literal es un nombre de variable (`At_3_2_0`) o su negación (`-At_3_2_0`). Es la forma en que se construye la base de conocimiento de RobotPhysics — mucho más eficiente para las KB grandes del resto de la práctica, con miles de cláusulas.

Todo el resto de la práctica (Preguntas 2 en adelante) trabaja **directamente con cláusulas**, sin pasar por `to_cnf`.

2.2. RobotPhysics: nombres de variables y sensor de contacto

Dado en `practica2_enunciado.py`. Convención de nombres de variables proposicionales (los cuatro símbolos con los que se construye toda la KB):

Función	Ejemplo	Significado
<code>at_symbol(x,y,t)</code>	<code>At_3_2_0</code>	el robot está en (3, 2) en el instante $t=0$
<code>action_symbol(a,t)</code>	<code>Act_N_0</code>	la acción tomada en $t=0$ es N
<code>blocked_symbol(d,t)</code>	<code>Blocked_N_0</code>	el sensor de contacto detecta muro al N en $t=0$
<code>detected_symbol(x,y,t)</code>	<code>Detected_3_2_0</code>	el objeto en (3, 2) ya ha sido detectado en $t=0$

`_successor_causes(x,y,t,world)` devuelve, para cada dirección por la que se podría haber llegado a (x,y) en el instante t , el par (símbolo de posición previa, símbolo de acción previa) que lo causaría — se usa en la Pregunta 3a.

`wall_contact_sensor_model(t,world,free_coords)` construye las cláusulas del sensor de contacto: para cada dirección d , $Blocked_d(t) \Leftrightarrow \bigvee_{c: \text{muro al lado de } c \text{ en dirección } d} At(c,t)$ — aquí `Wall(x,y)` no necesita ser una variable proposicional porque el mapa ya se conoce por completo (basta con mirar `world.is_wall`), a diferencia de la posición del robot, que sí es desconocida y por eso necesita una variable.

3. Desarrollo de la práctica

3.1. Paso 1: Ejecución inicial

Ejecuta el fichero tal cual se proporciona:

```
python3 practica2_enunciado.py
```

Debería generarse una casa y mostrarse con el robot colocado en ella (nada de lo que hay que implementar se llama todavía).

3.2. Paso 2: Pregunta 1 — calentamiento con Expr

Implementa tres fórmulas usando `Expr` y sus operadores, **sin simplificar** (tal cual se enuncian, sin aplicar equivalencias lógicas a mano):

```
sentence1() -> Expr:  $(A \vee B) \wedge (\neg A \Leftrightarrow (\neg B \vee C)) \wedge (\neg A \vee \neg B \vee C)$ 
sentence2() -> Expr:  $(C \Leftrightarrow (B \vee D)) \wedge (A \Rightarrow (\neg B \wedge \neg D)) \wedge (\neg(B \wedge \neg C) \Rightarrow A) \wedge (\neg D \Rightarrow C)$ 
sentence3() -> Expr: ver docstring en el código (modelo simple de un robot que se enciende)
```

Y dos funciones sobre fórmulas ya construidas:

```
entails(premise, conclusion) -> bool
pl_true_inverse(assignments, inverse_statement) -> bool
```

`entails` debe decidir si `premise` implica `conclusion` usando `to_cnf` y `find_model` (recuerda: $P \models C$ si y sólo si $P \wedge \neg C$ es insatisfacible). `pl_true_inverse` es un envoltorio de una línea sobre `pl_true`.

3.3. Paso 3: Pregunta 2 — bloques CNF reutilizables

Implementa, construyendo cláusulas **directamente** (sin `to_cnf`: con muchos literales, convertir a CNF por el camino general sería mucho más lento que escribir las cláusulas a mano):

```
at_least_one(literals) -> List[List[str]]
at_most_one(literals) -> List[List[str]]
exactly_one(literals) -> List[List[str]]
```

`exactly_one` se apoya en las otras dos. Estos tres bloques son la base de *toda* la KB del resto de la práctica: “el robot está en exactamente una celda”, “se toma exactamente una acción por instante”...

3.4. Paso 4: Pregunta 3 — axiomas de RobotPhysics

Tres funciones, de la más local a la más completa.

3a. `robot_successor_axiom_single(x,y,t,world) -> List[List[str]]`: las cláusulas de

$$At(x, y, t) \Leftrightarrow \bigvee_{(pos, acc) \in _successor_causes(x, y, t, world)} (pos \wedge acc)$$

El robot se mueve siempre (no hay acción de quedarse quieto, igual que en la Práctica 1): si no hay ninguna causa posible, $At(x, y, t)$ debe quedar forzado a falso. Cada causa es una conjunción de dos literales; distribuir la disyunción de conjunciones directamente en CNF sería exponencial en el número de causas (hasta 4, una por dirección). En su lugar, usa una **variable auxiliar por causa**: $aux_i \Leftrightarrow pos_i \wedge acc_i$ (3 cláusulas cada una), y luego $At(x, y, t) \Leftrightarrow \bigvee_i aux_i$ (lineal en el número de causas).

3b.

```
robot_physics_axioms(
    t, world, free_coords, *,
    sensor_model=None, include_successor=True
) -> List[List[str]]
```

combina, para el instante `t`: `exactly_one` sobre las posiciones de `free_coords`; las cláusulas de `sensor_model(t, world, free_coords)` si se ha dado uno; y `robot_successor_axiom_single` para cada celda si `include_successor` y `t >= 1`.

3c.

```
check_position_satisfiability(world, pos0, action0, query_pos)
-> (Optional[Model], Optional[Model])
```

dada la posición en $t=0$ y la acción tomada en $t=0$, ¿es posible que el robot esté en `query_pos` en $t=1$? ¿Es posible que NO lo esté? Construye la KB con `robot_physics_axioms` en $t=0$ (sin sucesor) y $t=1$ (con sucesor), fija `pos0` y `action0`, y llama `find_model` dos veces (añadiendo $At(query_pos, 1)$ una vez, su negación la otra).

`action0` se asume una acción **legal** desde `pos0`: al no haber acción de quedarse quieto, forzar una acción que choca con un muro deja la KB entera sin ningún $At(*, 1)$ posible y las dos respuestas salen `None` — no es un error del código, es la forma en que el modelo señala que el escenario planteado es contradictorio.

3.5. Paso 5: Pregunta 4 — planificación por SAT

Implementa:

```
robot_position_logic_plan(world, start, goal, max_t=50) -> List[str]
```

El análogo lógico de la Práctica 1: en vez de explorar un árbol de búsqueda, se construye la KB para $T = 0, 1, 2, \dots$ y se pregunta al solver si es satisfacible que el robot llegue a `goal` exactamente en el paso T . La primera vez que lo sea, se extrae la secuencia de acciones del modelo (usa `_extract_actions`, dado). Construye la KB de forma **incremental**: en la iteración de T , añade solo las cláusulas nuevas de ese instante a la KB acumulada de iteraciones anteriores, no la reconstruyas entera cada vez.

3.6. Paso 6: Pregunta 5 — catalogar objetos por SAT

Implementa:

```
catalog_logic_plan(world, start, targets, max_t=50) -> List[str]
```

Como la Pregunta 4, pero el objetivo es haber detectado todos los objetos de `targets` en algún momento hasta T (no llegar a un único punto): el mismo problema que la Pregunta 7 de la Práctica 1. Necesita una variable $Detected(x, y, t)$ por cada objetivo, con su propio axioma de sucesor:

$$Detected(x, y, t) \Leftrightarrow Detected(x, y, t - 1) \vee At(x, y, t)$$

(una vez detectado, se queda detectado). **Atención:** el segundo término es At en el instante *actual* t , no $t - 1$ — la detección se activa el mismo instante en que se llega a la celda, no un paso después; un desfase aquí hace que el plan encontrado sea siempre un paso más largo de lo necesario. A diferencia de la 3a, aquí no hace falta variable auxiliar: es una disyunción simple de 2 términos, se codifica directamente en 3 cláusulas. $Detected(x, y, 0)$ es falso salvo que $(x, y) = \text{start}$.

Nota de rendimiento. El solver de `sat_solver.py` es una implementación propia y educativa de DPLL, con propagación unitaria mediante *watched literals*, pero sin aprendizaje de cláusulas (CDCL). Para *un* objetivo se comporta tan bien como la Pregunta 4; para *varios* objetivos simultáneos, el problema introduce además distintas posibles secuencias de visita, por lo que el coste puede crecer mucho y de forma poco predecible, incluso con 3–4 objetivos. Diferencias de un orden de magnitud entre semillas con el mismo número de objetivos no son extrañas. Esta es una consecuencia de resolver la planificación mediante búsqueda SAT con un solver educativo, sin las técnicas avanzadas de los solvers SAT modernos, y proporciona un punto de comparación útil con la Pregunta 7 de la Práctica 1, donde A^* con una buena heurística puede manejar de forma más eficiente el crecimiento del espacio de búsqueda. Por ello, el `main()` de esta práctica limita la demo a los 2 objetos más cercanos al inicio, y el autograder prueba `catalog_logic_plan` solo con 1–2 objetivos en casas pequeñas.

3.7. Paso 7 (opcional): Pregunta 6 — localización por eliminación

Implementa (cuenta como puntuación extra):

```
localize_by_elimination(world, action_history, observation_history) ->
    Set[Coord]
```

Dada una secuencia de acciones ya ejecutadas y, para cada instante, la lectura del sensor de contacto con paredes (`wall_contact_sensor_model`), con posición inicial **desconocida**: ¿en qué celdas podría estar el robot ahora, según lo que la KB permite descartar? (`len(observation_history) == len(action_history) + 1`: una observación antes de la primera acción y una tras cada acción)

subsiguiente.) Para cada celda candidata, construye la KB completa (física con `sensor_model` puesto a `wall_contact_sensor_model` en cada instante, las acciones tomadas, las observaciones percibidas como literales fijos, y esa celda como posición de partida) y comprueba con `find_model` si es satisfacible; el resultado es el conjunto de celdas para las que lo es.

4. Ejemplo ilustrativo de pregunta del cuestionario

Por ejemplo: añadir un quinto símbolo de sensor (detecta objeto en la celda actual) y pedir explicar, sin ver el código, cómo tendría que cambiar `robot_physics_axioms` para incorporarlo, o razonar si la Pregunta 3c (`check_position_satisfiability`) seguiría teniendo sentido con ese sensor añadido. Esto no es la pregunta real, solo un ejemplo del estilo esperado.

5. Para reflexionar (no se entrega, pero puede salir en el examen)

Las Preguntas 4 y 5 resuelven los mismos problemas que las Preguntas 1–4 y 7 de la Práctica 1. Compara ambos enfoques: ¿qué algoritmo encuentra antes el plan en la práctica? ¿Cuál generaliza mejor a problemas con más de un objetivo (ver la nota de rendimiento de la Sección 3.6)? ¿Qué pasaría si el coste de moverse no fuera uniforme (como en la Pregunta 3 de la Práctica 1) — sería fácil de incorporar a la formulación lógica?

6. Entregables

- Un único fichero `practica2_enunciado.py` completado, renombrado a `practica2.py`, junto con `sat_solver.py` sin modificar, entregado en la tarea del aula virtual habilitada para ello.
- El código debe ser legible y estar debidamente tabulado. No es necesario comentarlo explicando su funcionalidad.